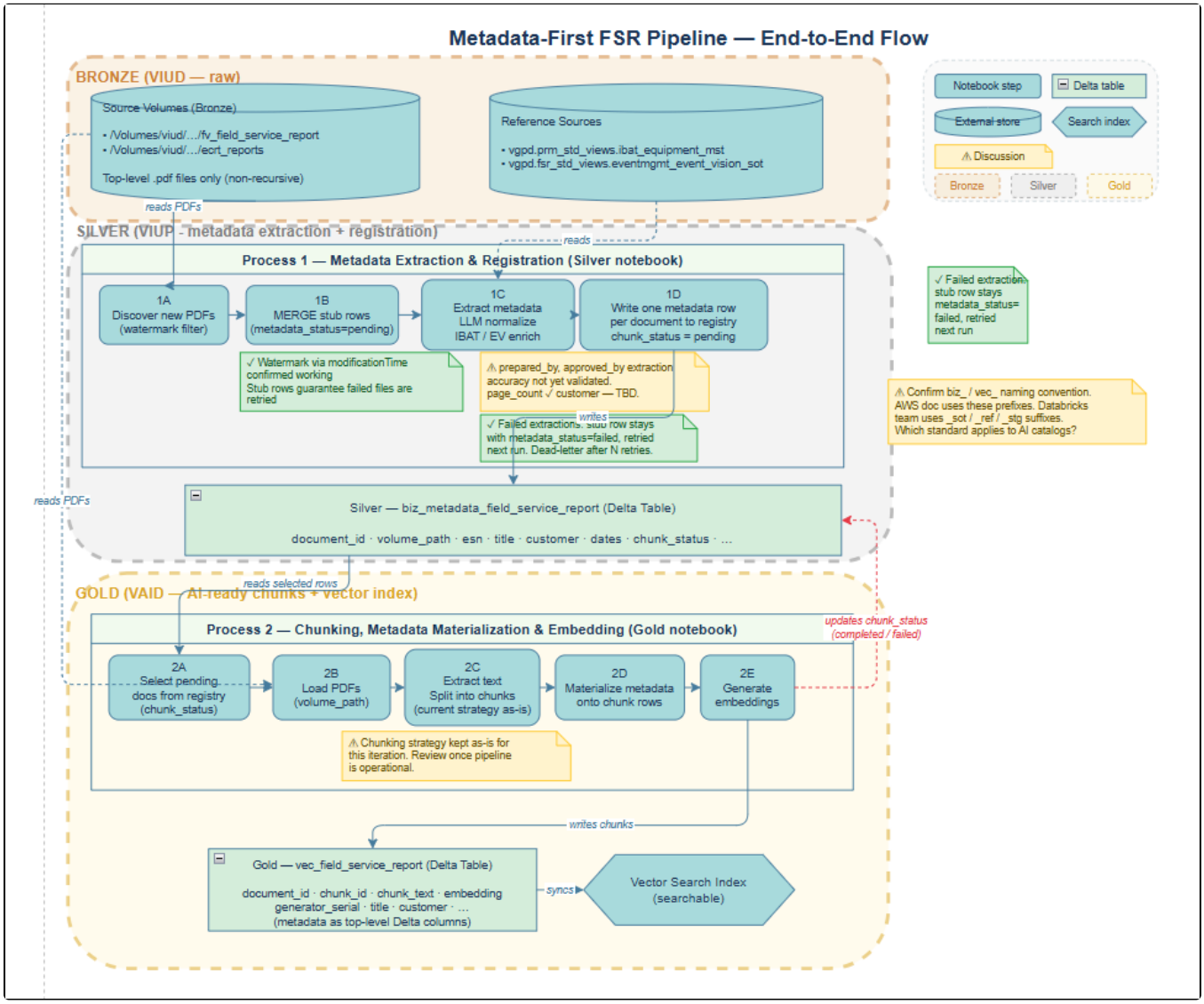
Scraping + Chunking – Design

This document describes the two core processes that together form the FSR (Field Service Report) data pipeline:

1. **Process 1 – Scraping & Metadata Extraction (Silver)**
2. **Process 2 – Chunking & Vector Embedding (Gold)**

Both processes run on Databricks and operate on the same set of PDF source volumes.

Architecture Overview



Process 1 – Scraping & Metadata Extraction

Purpose

Scan FSR PDF files from Unity Catalog volumes, extract structured metadata fields from each PDF, normalise them using an LLM, enrich with IBAT / Event Vision lookup tables, and write the results to a table.

Input

Source Volumes

Files are read non-recursively from the following Unity Catalog volume paths:

#	Volume Path	Catalog
1	`/Volumes/viud/ing_ud_fsr_manual/manual_field_service_report/ecrt_reports`	`viud`
2	`/Volumes/viud/ing_ud_fieldvision/fv_field_service_report`	`viud`

File Types

- **Accepted:** `.pdf` files at the top level of each volume path
- **Skipped:** `.crdownload`, `.tmp`, `.part`, `.download`, `.DS_Store`, `.json`, `.txt`, `.csv`, `.xlsx`, and any sub-directories

Lookup / Enrichment Sources

Source	Default Location	Purpose
IBAT Equipment	`vgpp.prm_std_views.ibat_equipment_mst`	ESN / equipment type enrichment
Event Vision SOT	`vgpp.fsr_std_views.eventmgmt_event_vision_sot`	Event / project ID enrichment
FSR PDF Reference	`vgpp.fsr_std_views.fsr_pdf_ref`	Resolve human-readable `pdf_name` from `document_id -> s3_filename`
PSOT	`vgpp.fsr_std_views.fsr_field_vision_field_services_report_psot`	Resolve `outage_type` and `technology_type` via `ev_equipment_event_id`

Processing Steps (high-level)

Step 1 – Discovery

1. Scan volumes via `dbutils.fs.ls`
2. Compare discovered paths against `volume_path` in the output table
3. Insert stub rows for new files with `metadata_status = 'pending'`; reset `metadata_status = 'pending'` for re-uploaded files (changed `file_last_modified`)

Step 2 – PDF Extraction

1. Query output table: `WHERE metadata_status = 'pending' OR metadata_status = 'failed'`
2. Extract key-value fields and title from page 1 of each PDF (`pdfplumber`)
3. Capture `page_count`, `file_size_bytes`, `file_last_modified` from the volume listing entry

Step 3 – LLM Normalisation

1. Send page 1 extracted text to LLM for field normalisation (model: `gemini-3-flash` by default) — produces `title`, `customer`, `esn`, `equipment_sys_id`, `event_type`, `ev_project_id`, `ev_equipment_event_id`, `ofs_event_id`, `fsp_project_id`, `xxx_project_id`, `fsr_number`, `report_issued_date`, `outage_start_date`, `outage_end_date`, `prepared_by`, `approved_by`

Step 4 – Enrichment

1. **IBAT** (`ibat_equipment_mst`): match on `equipment_sys_id` or `esn` fill `equipment_type`, `equipment_class_code`; backfill `esn` / `equipment_sys_id` if blank; set `esn_source = 'ibat'` if ESN was resolved here.
2. **Event Vision SOT** (`eventmgmt_event_vision_sot`): match on `ev_project_id` / `ev_equipment_event_id` / `ofs_event_id` / `fsp_project_id` backfill any blank event/project ID fields and `event_type`.
3. **FSR PDF Reference** (`fsr_pdf_ref`): match `document_id` `s3_filename` resolve human-readable `pdf_name`.
4. **PSOT** (`psot`): match `ev_equipment_event_id` resolve `outage_type` and `technology_type`

Step 5 – Write 13. Update the output table row in-place with all extracted and enriched fields:

- **On success** set `metadata_status = 'completed'`, `scraped_at = current_timestamp()`
- **On failure** set `metadata_status = 'failed'`, `metadata_error = <message>`, `scraped_at = current_timestamp()`

> ***FORCE_RESET override:*** *Setting `FORCE_RESET = true` truncates the output table and re-scans all files from scratch.*

Output

Output Table

`vaid/vaiq/vaip.ai_sot_field_service_report.biz_metadata_field_service_report`

Schema

Column	Type	Description
document_id	STRING NOT NULL	**Primary key.** UUID filename stem of the source PDF — also the s3_filename value in fsr_pdf_ref
pdf_name	STRING	Human-readable PDF name — resolved from fsr_pdf_ref by matching document_id to s3_fimename and picking pdf_name
volume_path	STRING NOT NULL	Full /Volumes/... path to the source PDF
title	STRING	Report title extracted from the PDF
customer	STRING	Customer / site name
esn	STRING	Equipment Serial Number (resolved via precedence)
esn_source	STRING	How ESN was resolved: llm / ibat / null
equipment_sys_id	STRING	System ID from IBAT
equipment_type	STRING	Equipment type (e.g. Generator, Gas Turbine)
equipment_class_code	STRING	Equipment sub-class / code
event_type	STRING	Event type (e.g. Call-Out, Inspection)
ev_project_id	STRING	Event Vision project ID
ev_equipment_event_id	STRING	Event Vision equipment event ID
ofs_event_id	STRING	OFS event ID
fsp_project_id	STRING	FSP project ID
xxx_project_id	STRING	Additional project ID (pipe-separated if multiple)
fsr_number	STRING	FSR document number
report_issued_date	STRING	Date the FSR was issued — **format: YYYY-MM-DD**
outage_start_date	STRING	Outage start date — **format: YYYY-MM-DD**
outage_end_date	STRING	Outage end date — **format: YYYY-MM-DD**
outage_type	STRING	Outage type — resolved from PSOT table via ev_equipment_event_id
technology_type	STRING	Technology type — resolved from PSOT table via ev_equipment_event_id
prepared_by	STRING	Name of the person who prepared the report
approved_by	STRING	Name of the person who approved the report
document_summary	STRING	LLM-generated full-document summary (3–5 sentences)
page_count	INT	Number of pages in the PDF
file_size_bytes	LONG	File size in bytes
file_last_modified	TIMESTAMP	Last modified timestamp from volume listing
metadata_status	STRING NOT NULL	Scraping status: pending / completed / failed
metadata_error	STRING	Error message if metadata extraction failed
chunk_status	STRING NOT NULL	Chunking status: pending / completed / failed
chunk_error	STRING	Error message if chunking failed
ingested_at	TIMESTAMP	When this row was first created (file discovery)
scraped_at	TIMESTAMP	When metadata scraping last completed or failed
chunked_at	TIMESTAMP	When chunking last completed or failed

> ***Date format:*** *All date columns must be normalised to `YYYY-MM-DD` (ISO 8601) during the LLM normalisation step, regardless of how they appear in the source PDF.*

Process 2 – Chunking & Vector Embedding

Purpose

Read FSR PDFs from the same volumes, parse and recursively chunk the full document content, generate vector embeddings, and write chunk rows to a Delta table backed by Databricks Vector Search.

Input

Source Table (Process 1 Output)

Process 2 does **not** scan volumes directly. It reads from the Process 1 output table and processes only rows where `chunk_status` is `pending`, or `failed`.

Table	Purpose
`vaid/vaiq/vaip.ai_sot_field_service_report.biz_metadata_field_service_report`	Provides both the `volume_path` to locate each PDF and all metadata fields for enrichment

The `volume_path` column on each row is used to load the physical PDF file from the volume. No separate volume scan or enrichment table is required — all metadata is already available in the source table row.

> *Because Process 2 filters on `chunk_status` status, re-running the job automatically picks up any new or previously failed files without any additional configuration.*

Processing Steps (high-level)

1. Read all rows from the Process 1 metadata table where `chunk_status` is `pending`, or `failed`
2. For each row, use `volume_path` to load the PDF file from the Unity Catalog volume
3. Extract full text from the PDF
4. Recursively split text into chunks
5. Denormalise metadata from the source row onto every chunk (no additional lookup needed)
6. Generate vector embeddings via LiteLLM (`azure-text-embedding-3-large-1`)
7. Write chunk rows to the Delta chunk table
8. **On success:** update the source row — set `chunk_status = 'completed'`
9. **On failure:** update the source row — set `chunk_status = 'failed'`, populate `error` and `error_reason`; the row will be retried on the next run
10. Sync Delta table to Databricks Vector Search index

Output

Output Delta Table

`vaid/vaiq/vaip.ai_std_con_field_service_report.vec_field_service_report`

Output Vector Search Index

`vaid/vaiq/vaip.ai_std_con_field_service_report.vs_vec_field_service_report`

Chunk Row Schema

Column	Type	Description
`chunk_id`	STRING NOT NULL	**Primary key.** Unique identifier per chunk — generated as `md5(pdf_name + "_" + chunk_index)`
`pdf_name`	STRING NOT NULL	FK to metadata registry — UUID filename stem
`page_number`	INT	Page number within the PDF where this chunk originates

chunk_text	STRING NOT NULL	Plain text content of the chunk
esn	STRING	Equipment Serial Number — top-level column for Vector Search filtering (renamed from generator_serial)
report_date	DATE	Report issued date (kept for backward compatibility)
chunk_embedding	ARRAY<DOUBLE>	Pre-computed embedding vector (3072 dimensions) — azure-text-embedding-3-large-1
metadata	STRING (JSON)	All contextual fields serialised as a JSON object — see metadata fields below
created_at	TIMESTAMP	When this chunk row was written

metadata JSON Fields

All fields below are written into the metadata column as a single JSON object at chunking time, sourced directly from the metadata registry row. esn and pdf_name are duplicated here for self-contained context when deserialising chunks on the client side.

Field	Description
pdf_name	Source PDF file name / identifier
page_number	Page number within the PDF where this chunk originates
title	Report title
customer	Customer / site name
esn	Equipment Serial Number
equipment_sys_id	System ID from IBAT
equipment_type	Equipment type (e.g. Generator, Gas Turbine)
equipment_class_code	Equipment sub-class / code
event_type	Event type (e.g. Call-Out, Inspection)
ev_project_id	Event Vision project ID
ev_equipment_event_id	Event Vision equipment event ID
ofs_event_id	OFS event ID
fsp_project_id	FSP project ID
xxx_project_id	Additional project ID (pipe-separated if multiple)
fsr_number	FSR document number
report_issued_date	Date the FSR was issued (YYYY-MM-DD)
outage_start_date	Outage start date (YYYY-MM-DD)
outage_end_date	Outage end date (YYYY-MM-DD)
outage_type	Outage type — resolved from PSOT table
technology_type	Technology type — resolved from PSOT table
prepared_by	Name of the person who prepared the report
approved_by	Name of the person who approved the report
document_summary	LLM-generated full-document summary (3–5 sentences)

Open Items / To Be Confirmed

- [ ] **LLM-generated document summary in metadata**